

Chapitre 4: Les classes et les objets en PHP

I. Introduction :

Depuis PHP 5, le modèle objet a été réécrit pour permettre de meilleures performances et offrir plus de fonctionnalités. Cette réécriture a été une des modifications majeures par rapport à PHP 4. PHP 5 a donc un modèle objet complet.

II. Notion de classe :

1. Introduction : qu'est-ce qu'une classe ?

Les classes sont, comme les fonctions, des dispositifs permettant la réutilisation du code, mais le système des classes est extrêmement plus sophistiqué. Les caractéristiques les plus importantes sont :

- Une "classe" est un ensemble d'éléments de programmes beaucoup plus complet qu'une simple fonction, elle englobe des déclarations, des `□` fonctions, des calculs... rassemblés dans une classe afin de réaliser une tâche spécifique. Lorsque plusieurs programmes séparés utilisent une même classe, ils utilisent en fait chacun une "instance" de cette classe, donc il n'y a pas de risque d'interférence entre ces différents programmes, principalement au niveau des variables, chacun pouvant manipuler indépendamment chaque variable de son instance de classe.
- Une classe peut faire appel à une autre classe et ainsi de suite (héritage), il y a souvent une notion de hiérarchie dans les classes.

2. Syntaxe de base :

Une définition de classe basique commence par le mot-clé ***class***, suivi du nom de la classe. Suit une paire d'accolades contenant la définition des propriétés et des méthodes. Le nom de la classe peut être quelconque à condition que ce ne soit pas un mot réservé en PHP. Une classe peut contenir ses propres constantes, variables (appelées "propriétés" ou "attributs"), et fonctions (appelées "méthodes").

Exemple 1 Définition typique d'une classe

```
<?php
class SimpleClass
{
    // déclaration des propriétés
    public $var = 'a default value';

    // déclaration des méthodes
    public function displayVar() {
        echo $this->var;
    }
}
?>
```

Exemple 2

```
<?php
class Personne
{
    // Attributs public
    $nom; public $prenom;
    public $dateDeNaissance;
    public $taille; public
    $sexe;
    // Constantes const
    NOMBRE_DE_BRAS = 2; const
    NOMBRE_DE_JAMBES = 2; const
    NOMBRE_DE_YEUX = 2; const
    NOMBRE_DE_PIEDS = 2; const
    NOMBRE_DE_MAINS = 2;

    // Méthodes
    public function __construct() { }

    public function boire()
    {
        echo 'La personne boit<br/>';
    }

    public function manger()
    {
        echo 'La personne mange<br/>';
    }
}
```

III. Les objets

1. Définition

Une fois qu'une classe a été définie, PHP vous permet d'engendrer autant d'instances de la classe que vous voudrez. Ces instances d'une classe sont dites des "**objets**". Chacune de ces instances est un objet complètement indépendant, avec ses propres propriétés et méthodes, et peut être remodelé ainsi indépendamment d'autres instances. Cette opération se fait à l'aide du mot clé *new* permettant de faire des objets découlant d'une classe. La syntaxe du mot clé *new* est la suivante :

```
$Nom_de_l_objet = new Nom_de_la_classe;
```

L'instanciation d'une classe est la phase de création des objets issus de cette classe. Lorsque l'on instancie une classe, on utilise le mot-clé *new* suivant du nom de la classe. Cette instruction appelle la méthode constructeur (`__construct()`) qui construit l'objet et le place en mémoire. Voici un exemple qui illustre 4 instances différentes de la classes *Personne*.

Création d'objets de type Personne

```
<?php

$personnel = new Personne();
?>
```

2. Accéder aux propriétés d'un objet

L'accès aux propriétés d'un objet se fait grâce au nom de l'objet, suivi d'une flèche (->) représentée par un moins (-) et un signe supérieur (>), puis du nom de la donnée membre (sans le signe \$). Par exemple :

\$Nom_de_l_objet->Nom_de_la_donnee_membre = Valeur;

3. Accéder aux méthodes d'un objet

L'accès aux méthodes d'un objet se fait comme pour l'accès aux propriétés, c'est-à-dire par le nom de l'objet, suivi d'une flèche et du nom de la méthode. La méthode est suivie de parenthèses, contenant les paramètres, si il y'en a. L'accès à une méthode se fait donc de la façon suivante :

\$Nom_de_l_objet->Nom_de_la_fonction_membre(parametre1,parametre2,..);

4. Accès aux constantes

L'accès aux constantes ne peut se faire qu'en lecture via l'opérateur :: . L'exemple suivant illustre la lecture d'une constante de la classe *Personne*.

Accès à une constante

```
<?php
echo 'Chaque personne a ', Personne::NOMBRE_DE_YEUX , ' yeux.';
?>
```

5. La variable courante \$this

Le mot clé *\$this* permet de désigner l'objet dans lequel on se trouve, c'est-à-dire que lorsque l'on désire faire référence dans une fonction membre à l'objet dans lequel elle se trouve, on utilise *\$this*. Grâce à cette variable spéciale, il est possible dans une fonction membre de faire référence aux propriétés situées dans le même objet que la fonction membre.

Exemple :

```
class Toto {
public $age;

    public $sexe;

    public $adresse;
```

```

        function DefineTotoAge($Age)
    {
        $this->age = $Age;
    }

```

3

```

    }
    $toto_test = new Toto;

    $toto_test->DefineTotoAge(10);
    echo "L'age de TOTO : " . $toto_test->age .
    "<br/>";

```

IV. Visibilité des propriétés et des méthodes

PHP5 introduit les 3 niveaux de visibilité :

- *public* : Les attributs et méthodes sont visibles de n'importe où, comme dans PHP4.
- *private* : Les attributs et méthodes ne sont visibles qu'au sein même de la classe où ils sont définis.
- *protected* : Les attributs et méthodes sont visibles à la fois au sein de la classe et dans chaque classe héritant de celle-ci.

1. Visibilité des variables de classe

Les variables de classes **protégées** peuvent être accédées par les classes dérivées des classes dans lesquelles elles sont protégées, tandis que les variables de classes **privées** ne peuvent être utilisées que dans les classes dans lesquelles elles sont déclarées (sauf accès direct avec l'opérateur ::).

Exemple :

```

<?php
/**
 * Définition de MyClass
 */
class MyClass
{ public $public = 'Public';
  protected $protected = 'Protected';
  private $private = 'Private';

  function printHello()
  { echo $this-
    >private; echo $this-
    >protected; echo
    $this->private;
  }
}

```

```

$obj = new MyClass(); echo $obj-
>public; // Fonctionne echo $obj-
>protected; // Erreur fatale echo
$obj->private; // Erreur fatale
$obj->printHello(); // Affiche Public, Protected et Private

```

```

/**
 * Définition de MyClass2

```

4

```

 */
class MyClass2 extends MyClass
{
    // On peut redéclarer les éléments publics ou protégés,
    mais pas les privés
    protected $protected = 'Protected2';

    function printHello()
    { echo $this-
    >public; echo $this-
    >protected; echo
    $this->private;
    }
}

$obj2 = new MyClass2(); echo $obj-
>public; // Fonctionne echo $obj2-
>private; // Indéfini echo $obj2-
>protected; // Erreur fatale
$obj2->printHello(); // Affiche Public, Protected2 et non
Private

?>

```

2. Visibilité des méthodes

Le concept est le même que pour les variables (voir ci-dessus) .

Exemple :

```

<?php
/**
 * Définition de MyClass
 */ class
MyClass
{
    // Les constructeurs doivent être publics
    public function __construct() { }

    // Déclaration d'une méthode publique
    public function MyPublic() { }

```

```

    // Déclaration d'une méthode protégée
protected function MyProtected() { }

    // Déclaration d'une méthode privée
private function MyPrivate() { }

    // Celle-ci sera publique
function Foo()
{
    $this->MyPublic();
    $this->MyProtected();
    $this->MyPrivate();
}

5

}

$myclass = new MyClass;
$myclass->MyPublic(); // Fonctionne
$myclass->MyProtected(); // Erreur fatale
$myclass->MyPrivate(); // Erreur fatale
$myclass->Foo(); // Public, Protected et Private fonctionnent

/**
 * Définition de MyClass2
 */
class MyClass2 extends MyClass
{
    // Celle-ci sera publique
function Foo2()
{
    $this->MyPublic();
    $this->MyProtected();
    $this->MyPrivate(); // Erreur fatale
}
}

$myclass2 = new MyClass2;
$myclass2->MyPublic(); // Fonctionne
$myclass2->Foo2(); // Public et Protected fonctionnent, non pas
Private
?>

```

V. Constructeurs et destructeurs



Constructeurs/destructeurs : comme indiqué ci-dessus l'appel de la classe provoque l'exécution du constructeur. Par contre le destructeur doit être appelé spécifiquement et il permet de libérer la partie mémoire utilisée par l'objet lorsque celui-ci n'est plus nécessaire.

1. Constructeurs : ils peuvent être appelés par le mot-clé `__construct()`
2. Destructeurs : ils peuvent être appelés par le mot-clé `__destruct()`

1. Constructeurs

void **__construct** ([[mixed](#) \$args [, \$...]])

PHP permet aux développeurs de déclarer des constructeurs pour les classes. Les classes qui possèdent une méthode constructeur appellent cette méthode à chaque création d'une nouvelle instance de l'objet, ce qui est intéressant pour toutes les initialisations dont l'objet a besoin avant d'être utilisé.

Note: Les constructeurs parents ne sont pas appelés implicitement si la classe enfant définit un constructeur. Si vous voulez utiliser un constructeur parent, il sera nécessaire de faire appel à *parent::__construct()*.

Exemple d'utilisation des nouveaux constructeurs unifiés

6

```
<?php
class BaseClass {
    function __construct() {
        print "In BaseClass constructor\n";
    }
}
class SubClass extends BaseClass {
    function __construct() {
        parent::__construct();
        print "In SubClass constructor\n";
    }
}
$obj = new BaseClass();
$obj = new SubClass();
?>
```

2. Destructeur :

void **__destruct** (void)

PHP 5 introduit un concept de destructeur similaire aux autres langages orientés objet, comme le C++. La méthode destructeur est appelée dès qu'il n'y a plus d'autres références sur des objets, ou dans n'importe quel ordre de la séquence d'arrêt.

Exemple avec un Destructeur

```
<?php
class MyDestructableClass {
    function __construct() {
        print "In constructor\n";
        $this->name = "MyDestructableClass";
    }
    function __destruct() {
        print "Destruction de " . $this->name . "\n";
    }
}
$obj = new MyDestructableClass();
?>
```

VI.L'héritage

On dit qu'une classe hérite d'une autre classe lorsqu'elle possède en elle tous les composants (méthodes, variables...) d'une autre classe. Contrairement au système de fonctions qui permet d'appeler une fonction à l'intérieur d'une autre fonction, avec la syntaxe normale d'appel de fonction, le système d'héritage fonctionne au moyen d'un mot-clé spécial **extends** à utiliser uniquement lors de la déclaration de la classe. Ainsi lorsqu'on veut mettre en place une classe qui hérite d'une autre classe, on va écrire :

```
<?php
class nouvelleClasse extends ancienneClasse
{
    ....}
?>
```

La classe **nouvelleClasse** va hériter de tous les composants de la classe **ancienneClasse**. Tous les constructeurs des classes parentes vont s'exécuter lors de l'instanciation de la classe fille :

7

Un exemple très simple pour illustrer l'héritage

```
<?php
class foo
{
    public function printItem($string)
    {
        echo 'Foo: ' . $string ;
    }

    public function printPHP()
    {
        echo 'PHP est super';
    }
}
```

```
class ba extends foo
{
    public function printItem($string)
    {
        echo 'Ba: ' . $string ;
    }
}
```

```
$foo1 = new foo();
$ba1 = new ba();
$foo1->printItem('baz'); // Affiche: 'Foo: baz'
$foo1->printPHP(); // Affiche: 'PHP est super'
$ba1->printItem('baz'); // Affiche: 'Ba: baz'
$ba1->printPHP(); // Affiche: 'PHP est super'
?>
```

VII. L'opérateur ::

L'opérateur **::** est utilisé à la place de **->** pour se référer à des fonctions et variables de la classe de base ou à des fonctions qui appartiennent à des classes qui n'ont pas encore d'instances. Dans l'exemple ci-après, on appelle la fonction `example()`, mais il n'y a

aucun objet créé à partir de la classe A, aussi nous ne pouvons pas écrire \$a>example(), c'est pourquoi nous appelons example() comme une 'fonction de classe', c'est à dire une fonction de la classe elle-même, et pas un objet résultant de l'instanciation de la classe.

Une "fonction de classe" ne peut utiliser aucune variable d'objet (ne peut utiliser \$this)
La classe B redéfinit la fonction example(), la définition originale (classe A) devient cachée et n'est plus disponible, à moins que vous n'utilisiez l'opérateur :: pour y accéder. **Exemple:**

```
<!-------Début de l'exemple-----><?php class
A
{ function example()
{
echo "Je suis dans la classe de base A.<br/>\n"; }

8

} class B extends A {
function example()
{
echo "Je suis dans la classe B qui étend A .<br/>\n";
A::example();
}
}
//il n'y a pas d'objet de classe A. A::example();
// je crée un objet de classe B.
$b = new B;
$b->example();
?>
```

VIII. propriétés ou méthodes Statiques

Le fait de déclarer des membres ou des méthodes comme statiques vous permet d'y accéder sans avoir besoin d'instancier la classe. Un membre déclaré comme statique ne peut être accédé avec l'objet instancié d'une classe (bien qu'une méthode statique le peut).

La déclaration static doit être faite après la déclaration de visibilité. Pour des raisons de compatibilité avec PHP 4, si aucune déclaration de visibilité n'est utilisée, alors le membre ou la méthode sera traité comme s'il avait été déclaré comme public . Comme les méthodes statiques sont appelables sans instance d'objet créée, la pseudo variable \$this n'est pas disponible dans la méthode déclarée en tant que statique.

Si l'appel est effectuée par le mot clé self , alors self est traduit en la classe courante, qui est la classe appartenant au code. Ici , aucune notion d'héritage n'est appliquée. On ne peut pas accéder à des propriétés statiques à travers l'objet en utilisant l'opérateur -> .

L'appel non-statique à des méthodes statiques génère une alerte de degré E_STRICT .

Exemple avec un membre statique

```
<?php class
Foo {
public static $my_static = 'foo';
```

```

    public function staticValue()
    {
        return self::$my_static;
    } }
class Boo extends Foo
{ public function fooStatic() {
return parent::$my_static;
} }
print Foo::$my_static . "\n";
$foo = new Foo();
print $foo->staticValue() . "\n";
print $foo->my_static . "\n"; // propriété my_static non définie //
$foo::my_static n'est pas possible

?>

```

Exemple avec une méthode statique

9

```

<?php
class Foo
{ public static function aStaticMethod() {
// ...
}
}

Foo::aStaticMethod();

?>

```

VIII. Constantes de classe

Il est possible de définir des valeurs constantes à l'intérieur d'une classe, qui ne seront pas modifiables. Les constantes diffèrent des variables normales du fait qu'on n'utilise pas le symbole \$ pour les déclarer ou les utiliser. Tout comme pour les membres statiques, on ne peut pas accéder aux valeurs constantes depuis une instance de l'objet.

Exemple :

Définition et utilisation d'une constante de classe

```

<?php class
MyClass
{ const constant = 'valeur constante';
function showConstant() { echo
self::constant . "\n";
} } echo MyClass::constant .
"\n";
$class = new MyClass();
$class->showConstant();
// echo $class::constant; n'est pas autorisé

?>

```

IX. Classes et méthodes abstraites

Les classes abstraites sont des classes qui ne peuvent pas être instanciées. Elles peuvent donc être seulement "étendues" au moyen de l'héritage. Une méthode abstraite ne peut être déclarée que dans une classe abstraite et ne peut pas contenir de code mais seulement des commentaires (signature). Une classe qui contient une méthode abstraite doit être déclarée abstraite elle-même

Exemple :

```
<?php
abstract class AbstractClass
{
    abstract public function test();
}
class ImplementedClass extends AbstractClass
{
    public function test()
    {
        echo "Appel de la classe ImplementedClass.<br/>";
    }
}
```

10

```
}
}

$o = new ImplementedClass;

$o->test();
?>
```

X. Les interfaces

Une classe abstraite peut contenir des méthodes abstraites et des méthodes nonabstraites. Une interface est une classe qui ne contient que des méthodes abstraites. Les interfaces sont donc toujours abstraites, sans qu'il soit nécessaire de l'indiquer; il n'est pas non plus nécessaire de déclarer leurs méthodes comme étant abstraites. Dans le cas des interfaces, le mot-clé "implements" , qui remplace "extends", peut être suivi de plusieurs noms d'interfaces, contrairement à "extends" qui ne peut être suivi que d'un seul nom de classe

Exemple :

```
<?php interface
Foo
{
    function a();
}
interface Ba
{
```

```

function b();
}
class FooBa implements Foo, Ba
{
    function a()
    {
        print "aaaaaaa<br/>";
    }
    function b()
    {
        print "bbbbbbb<br/>";
    }
}
$aa = new FooBa;
echo "<b>Appel fonction a</b><br/>";
$aa->a();
echo "<b>Appel fonction b</b><br/>";
$aa->b();
?>

```

XI. Méthodes magiques

Toutes les méthodes dont les noms commencent par deux caractères de soulignement `__` sont des méthodes dites « magiques », dont la création est réservée à PHP et qui ont des rôles particuliers. Nous en avons déjà rencontré deux : `__construct()` et `__destruct()`,

11

respectivement constructeur et destructeur, appelées automatiquement lors de la création ou la destruction d'un objet. PHP en comporte d'autres qui sont appelées automatiquement dans des circonstances données, à savoir :

- `__set()` est exécutée quand le script tente de modifier une propriété déclarée `private` ou `protected`.
- `__get()` est exécutée lorsque le script tente de lire une propriété déclarée `private` ou `protected`.
- `__isset()` est exécutée quand le script tente de vérifier, avec les fonctions `isset()` ou `empty()`, si une propriété déclarée `private` ou `protected` existe ou est vide.
- `__unset()` est exécutée quand le script tente de supprimer une propriété déclarée `private` ou `protected` avec la fonction `unset()`.

L'exemple suivant fournit des applications de ces méthodes. La classe `maclasse` possède une propriété `code` déclarée `private`, donc normalement inaccessible à partir d'un objet. Le constructeur permet d'affecter une valeur à cette propriété. Les méthodes magiques `__set()`, `__get()`, `__isset()` et `__unset()` créent chacune une action qui est déclenchée respectivement quand le script affecte une valeur à la propriété `code`, tente de la lire, vérifie son existence ou veut la supprimer. Sans la définition de ces méthodes, toutes ces actions déclencheraient une erreur.

Notez que les méthodes définies ici autorisent ces opérations, mais que leur `code` pourrait afficher des messages interdisant toute modification de la propriété `code` qui est privée.

Exemple: Mise en oeuvre des méthodes magiques

```
<?php class
maclasse
{
private $code ;
public function __construct($val)
{
$this->code = $val;
}
public function __set($prop, $val)
{
echo "Affectation de la valeur $val à la propriété $prop <br /> ";
$this->$prop = $val;
}
public function __get($prop)
{
return $this->$prop;
}
public function __isset($prop)
{
if(isset($this->$prop)) echo "La propriété $prop est définie<br />"; else
echo "La propriété $prop n'est pas définie <br />";
}
public function __unset($prop)
{

```

12

```
echo "Effacement de la prop $prop <br />"; unset($this->$prop);
}
}
//Création d'un objet
$obj = new maclasse('AZERTY');
echo isset($obj->code); echo "code =
", $obj->code," <br />"; $obj->code="QWERTY";
echo "code = ", $obj->code," <br />";
echo isset($obj->code); unset($obj->code); echo "code = ". $obj->code,"
```

XIII. Mot clé "final"

PHP 5 introduit le mot-clé " final " qui empêche les classes filles de surcharger une méthode en en préfixant la définition par le mot-clé " final ". Si la classe elle-même est définie comme finale, elle ne pourra pas être étendue. **Exemple de méthode finale**

```
<?php class BaseClass {
public function test() {
echo "BaseClass::test() appelé\n";
}
```

```

    final public function moreTesting() {
echo "BaseClass::moreTesting() appelé\n";
    }
} class ChildClass extends
BaseClass { public function
moreTesting() {
    echo "ChildClass::moreTesting() appelé\n";
    }
}
// Résultat : Fatal error: Cannot override final method
BaseClass::moreTesting()
?>

```

Exemple de classe finale

```

<?php
final class BaseClass {
public function test() {
    echo "BaseClass::test() appelé\n";
}

// Ici, peut importe si vous spécifié la fonction en final ou pas
final public function moreTesting() { echo
"BaseClass::moreTesting() appelé\n";
}}

class ChildClass extends BaseClass {
}
// Résultat : Fatal error: Class ChildClass may not inherit from
final class (BaseClass) ?>

```

XIV. Clonage d'objets

Le fait de créer une copie d'un objet possédant exactement les mêmes propriétés n'est pas toujours le comportement que l'on souhaite.

Une copie d'objet est créée en utilisant le mot clé clone (qui fait appel à la méthode __clone de l'objet, si elle a été définie). La méthode __clone d'un objet ne peut être appelée directement.

```

<?php

$copie_d_objet = clone $objet;

?>

```

Lorsqu'un objet est cloné, PHP 5 effectue une copie superficielle de toutes les propriétés de l'objet. Toutes les propriétés qui sont des références à d'autres variables demeureront des références. Si une méthode __clone() est définie, la méthode

__clone() du nouvel objet sera appelée pour permettre à chaque propriété qui doit l'être d'être modifiée.

Exemple de duplication d'objets

```
<?php
class SubObject
{ static $instances =
0; public $instance;
public function __construct() {
$this->instance = ++self::$instances;
}

public function __clone() {
$this->instance = ++self::$instances;
} }
class MyCloneable
{ public
$objet1; public
$objet2;

function __clone()
{
// Force la copie de this->object, sinon
// il pointera vers le même objet.
$this->object1 = clone($this->object1);
}
}

$obj = new MyCloneable();

$obj->object1 = new SubObject();
$obj->object2 = new SubObject();
```

14

```
$obj2 = clone $obj;
print("Objet original :\n");
print_r($obj); print("Objet
cloné :\n"); print_r($obj2);
?>
```

EXERCICES

Exercice1

Nous voulons créer une classe qui contienne un attribut : \$texte, par défaut à 'Hello world !', et une méthode : changerTexte(). Cette méthode, à chaque appel, doit changer la valeur de l'attribut \$texte. Si celui-ci vaut 'Hello world !', alors l'attribut devra avoir pour valeur 'Bonjour tout le monde !', et vice-versa. Pas très utile, certes, mais ça fait pratiquer.

Code : PHP

```
<?php
```

```

1 class
  CH_TEXTE
2
  {
3 private $texte = 'Hello world
  !'; // L'attribut $texte que
4 l'on va changer.
5
6 public function
  changerTexte()
7
  {
8 if ($this->texte == 'Hello world
  !')
9
  $this->texte = 'Bonjour tout le monde !'; 10
  else
11
  $this->texte = 'Hello world !';
12
  }
13
  }
14 ?>

```

Exercice2

Classe de connexion à MySQL en PHP5 :

```

1. Class Mysql
2.
3. { private
4. $Serveur = '', 5. $Bdd
   = '', 6. $Identifiant = '',
7. $Mdp = '',
8. $Lien = '',
9. $Debogue = true,
10. $NbRequetes = 0;
   11. Public function __construct($Serveur = 'localhost',
   $Bdd = 'base', $Identifiant = 'root', $Mdp = '') 12. {
   $this->Serveur = $Serveur; 13. $this->Bdd = $Bdd; 14.
   $this->Identifiant = $Identifiant; 15. $this->Mdp =
   $Mdp; 16. $this->Lien=mysql_connect($this->Serveur,
   $this->Identifiant, $this->Mdp);
   if (!$this->Lien && $this->Debogue) throw new Erreur
   ('Erreur de connexion au serveur MySql!!!');

```

```
17. $Base = mysql_select_db($this-  
    >Bdd,$this->Lien);  
18. if (!$Base && $this->Debogue) throw new Erreur  
('Erreur de connexion à la base de donnees!!!');  
19.  
20. }  
21. Public function RetourneNbRequetes() 22.
```

```

23.         {return $this->NbRequetes;
24.
25.         }
26.     Public function TabResSQL($Requete)
27.
28.         {      $i = 0;
29.
30.         $Ressource = mysql_query($Requete,$this->Lien) ;
31.
32.         $TabResultat=array() ;
33.
34.         if (!$Ressource and $this->Debogue) throw new
Erreur ('Erreur de requête SQL!!!');
35.
36.         while ($Ligne = mysql_fetch_assoc($Ressource))
37.
38.             {
39.                 foreach ($Ligne as $clef => $valeur)
$TabResultat[$i][$clef] = $valeur;
40.
41.                 $i++;
42.             }
43.         mysql_free_result($Ressource) ;
44.
45.         $this->NbRequetes++;
46.
47.         return $TabResultat;
48.     }
49.     Public function DernierId()
50.
51.     {
52.         return mysql_insert_id($this->Lien) ;
53.
54.     }
55.     Public function ExecuteSQL($Requete)
56.
57.     {
58.         $Ressource = mysql_query($Requete,$this->Lien) ;
59.
60.         if (!$Ressource and $this->Debogue) throw new
Erreur ('Erreur de requête SQL!!!');
61.
62.         $this->NbRequetes++;
63.         $NbAffectee = mysql_affected_rows() ;
64.         return $NbAffectee;
65.
66.     }
67.
68. }
69. ?>

```